

06_pca_kmeans

June 24, 2026

1 Step 6: Subject-Level Geometry and Exploratory K-Means Views (Authoritative v4)

The machine-readable Step 6 summary remains **subject-level** and keeps the four required views: `disease_only`, `all_subjects`, `binary_pathological_vs_control`, and `control_a_vs_control_b`. This notebook is exploratory by design and should be read as geometry context rather than as supervised evidence.

```
[1]: import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import display
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score, calinski_harabasz_score,
↳davies_bouldin_score, silhouette_score
from sklearn.preprocessing import StandardScaler

def find_repo_root() -> Path:
    candidates = [Path.cwd().resolve(), *Path.cwd().resolve().parents]
    for base in candidates:
        if (base / 'data' / 'processed' / 'v4' / 'v4_protocol_manifest.json').
↳exists():
            return base
        raise RuntimeError('Could not locate repository root from the current_
↳working directory.')

REPO_ROOT = find_repo_root()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
if str(REPO_ROOT / 'src') not in sys.path:
```

```

sys.path.insert(0, str(REPO_ROOT / 'src'))

from src.features import ALL_FEATURE_COLS
from src.v4_provenance import collect_package_versions, sha256_file

STEP_TAG = 'step6'
FIG_PDF = REPO_ROOT / 'report' / 'figures' / 'v4' / 'pdf' / STEP_TAG
FIG_PNG = REPO_ROOT / 'report' / 'figures' / 'v4' / 'png' / STEP_TAG
TABLE_DIR = REPO_ROOT / 'report' / 'tables' / 'v4'
for directory in (FIG_PDF, FIG_PNG, TABLE_DIR):
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    'font.family': 'DejaVu Serif',
    'font.size': 9,
    'axes.labelsize': 9,
    'axes.titlesize': 10,
    'xtick.labelsize': 8,
    'ytick.labelsize': 8,
    'legend.fontsize': 7,
    'figure.dpi': 180,
    'savefig.bbox': 'tight',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
})

IEEE_ONE_COL = 3.5
IEEE_TWO_COL = 7.16
FEATURE_COLS = list(ALL_FEATURE_COLS)
PROC_DIR = REPO_ROOT / 'data' / 'processed' / 'v4'
FEATURE_PATH = PROC_DIR / 'gait_features_v4_subject_level.csv'
STRIDE_FEATURE_PATH = PROC_DIR / 'gait_features_v4.csv'
PROTOCOL_PATH = PROC_DIR / 'v4_protocol_manifest.json'
PREPROC_PATH = PROC_DIR / 'preprocessing_manifest_v4.json'
DOWNSTREAM_PATH = PROC_DIR / 'v4_downstream_execution_manifest.json'
PARTITION_PATH = PROC_DIR / 'control_partition_v4.json'
SUMMARY_JSON = REPO_ROOT / 'experiments' / 'results' / 'v4' / '
↳ 'step6_subject_views_summary.json'
NOTEBOOK_PATH = REPO_ROOT / 'notebooks' / '06_pca_kmeans.ipynb'

PROTOCOL = json.loads(PROTOCOL_PATH.read_text())
PREPROC = json.loads(PREPROC_PATH.read_text())
DOWNSTREAM = json.loads(DOWNSTREAM_PATH.read_text())
PARTITION = json.loads(PARTITION_PATH.read_text())
subject_df = pd.read_csv(FEATURE_PATH)
stride_df = pd.read_csv(STRIDE_FEATURE_PATH)

```

```

COND_ORDER = ['pd', 'hd', 'als', 'control']
COND_LABEL = {'pd': 'PD', 'hd': 'HD', 'als': 'ALS', 'control': 'Healthy_
↳control'}
COND_COLOR = {'pd': '#1f77b4', 'hd': '#2ca02c', 'als': '#ff7f0e', 'control':_
↳'#7f7f7f'}
RANDOM_SEEDS = {'pca': 42, 'kmeans': list(range(10)), 'bootstrap': 101}
STANDARDIZATION_RULE = 'z-score across subjects within each active view before_
↳PCA and K-means'

def savefig_stem(fig: plt.Figure, stem: str) -> None:
    pdf_path = FIG_PDF / f'{stem}.pdf'
    png_path = FIG_PNG / f'{stem}.png'
    fig.savefig(pdf_path)
    fig.savefig(png_path, dpi=300)
    print(f'Saved figure: {pdf_path.relative_to(REPO_ROOT)}')
    print(f'Saved figure: {png_path.relative_to(REPO_ROOT)}')

def save_table_csv(df: pd.DataFrame, name: str, *, index: bool = False) -> None:
    path = TABLE_DIR / name
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

def save_csv_path(df: pd.DataFrame, path: Path, *, index: bool = False) -> None:
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

def save_json_payload(payload, path: Path) -> None:
    path.write_text(json.dumps(payload, indent=2))
    print(f'Wrote summary: {path.relative_to(REPO_ROOT)}')

```

1.1 Exploratory PCA and K-means outputs

This cell writes the full Step 6 figure set plus the machine-readable summary JSON. The saved outputs are useful because they separate several related but different questions. The first two principal components already capture a substantial share of variance in the main views: about **0.699** for `disease_only`, **0.726** for `all_subjects`, and **0.721** for `control_a_vs_control_b`. The scatter and clustering plots are therefore visually informative, but the clustering agreement with the relevant labels remains weak enough that the notebook must stay exploratory: ARI is only **0.0135** in `disease_only`, **0.0260** in `all_subjects`, and effectively **0.0000** for `control_a_vs_control_b`.

```

[2]: def build_view(name: str):
    df = subject_df.copy()
    if name == 'disease_only':
        df = df[df['condition'].isin(['pd', 'hd', 'als'])].copy()

```

```

        labels = df['condition'].to_numpy()
        ari_labels = labels
    elif name == 'all_subjects':
        labels = df['condition'].to_numpy()
        ari_labels = labels
    elif name == 'binary_pathological_vs_control':
        df = df.copy()
        df['binary_label'] = np.where(df['condition'] == 'control', 'control',
        ↪ 'pathological')
        labels = df['binary_label'].to_numpy()
        ari_labels = labels
    elif name == 'control_a_vs_control_b':
        control_a = set(PARTITION['control_A'])
        df = df[df['condition'] == 'control'].copy()
        df['control_role'] = np.where(df['subject_id'].isin(control_a),
        ↪ 'control_A', 'control_B')
        labels = df['control_role'].to_numpy()
        ari_labels = labels
    else:
        raise ValueError(name)
    return df.reset_index(drop=True), labels, ari_labels

def evaluate_view(view_name: str):
    df, labels, ari_labels = build_view(view_name)
    X = df[FEATURE_COLS].to_numpy(dtype=float)
    scaler = StandardScaler()
    Xz = scaler.fit_transform(X)
    pca = PCA(n_components=min(len(FEATURE_COLS), Xz.shape[0]),
    ↪ random_state=RANDOM_SEEDS['pca'])
    Xp = pca.fit_transform(Xz)

    coords_path = TABLE_DIR / f'step6_{view_name}_pca_coordinates.csv'
    coords_df = pd.DataFrame({'subject_id': df['subject_id'], 'pc1': Xp[:, 0],
    ↪ 'pc2': Xp[:, 1], 'label': labels})
    save_csv_path(coords_df, coords_path)

    metric_rows = []
    best_k = None
    best_silhouette = None
    best_clusters = None
    for k in range(2, min(10, len(df) - 1) + 1):
        km = KMeans(n_clusters=k, random_state=RANDOM_SEEDS['pca'], n_init=20)
        clusters = km.fit_predict(Xz)
        silhouette = float(silhouette_score(Xz, clusters))
        metric_rows.append({
            'k': k,

```

```

        'silhouette_score': silhouette,
        'inertia': float(km.inertia_),
        'calinski_harabasz_score': float(calinski_harabasz_score(Xz,
↪clusters)),
        'davies_bouldin_score': float(davies_bouldin_score(Xz, clusters)),
        'ari_against_relevant_labels':
↪float(adjusted_rand_score(ari_labels, clusters)),
    })
    if best_silhouette is None or silhouette > best_silhouette:
        best_silhouette = silhouette
        best_k = k
        best_clusters = clusters
    metrics_df = pd.DataFrame(metric_rows)
    save_csv_path(metrics_df, TABLE_DIR / f'step6_{view_name}_kmeans_metrics.
↪csv')

    seed_aris = []
    for seed in RANDOM_SEEDS['kmeans']:
        km = KMeans(n_clusters=best_k, random_state=seed, n_init=20)
        seed_aris.append(adjusted_rand_score(best_clusters, km.fit_predict(Xz)))

    bootstrap_aris = []
    rng = np.random.default_rng(RANDOM_SEEDS['bootstrap'])
    for _ in range(25):
        idx = rng.integers(0, len(df), len(df))
        if len(np.unique(idx)) < best_k:
            continue
        km = KMeans(n_clusters=best_k, random_state=RANDOM_SEEDS['pca'],
↪n_init=20)
        clusters = km.fit_predict(Xz[idx])
        bootstrap_aris.append(adjusted_rand_score(np.asarray(ari_labels)[idx],
↪clusters))

    cluster_comp = pd.DataFrame({'cluster': best_clusters, 'label': labels}).
↪groupby(['cluster', 'label']).size().unstack(fill_value=0).to_dict()
    return {
        'subject_count': int(len(df)),
        'active_feature_list': FEATURE_COLS,
        'standardization_rule': STANDARDIZATION_RULE,
        'pca_explained_variance': [float(x) for x in pca.
↪explained_variance_ratio_[:5]],
        'pca_coordinates_path': str(coords_path.relative_to(REPO_ROOT)),
        'k_range_evaluated': [int(x) for x in metrics_df['k'].tolist()],
        'silhouette_score': float(metrics_df.loc[metrics_df['k'] == best_k,
↪'silhouette_score'].iloc[0]),

```

```

        'inertia': float(metrics_df.loc[metrics_df['k'] == best_k, 'inertia'].
↪iloc[0]),
        'calinski_harabasz_score': float(metrics_df.loc[metrics_df['k'] ==
↪best_k, 'calinski_harabasz_score'].iloc[0]),
        'davies_bouldin_score': float(metrics_df.loc[metrics_df['k'] == best_k,
↪'davies_bouldin_score'].iloc[0]),
        'ari_against_relevant_labels': float(metrics_df.loc[metrics_df['k'] ==
↪best_k, 'ari_against_relevant_labels'].iloc[0]),
        'seed_stability_mean_ari': float(np.mean(seed_aris)),
        'subject_bootstrap_stability_mean_ari': float(np.mean(bootstrap_aris))
↪if bootstrap_aris else None,
        'cluster_composition': cluster_comp,
        'cautious_interpretation_metadata': {
            'exploratory_only': True,
            'not_for_confirmatory_hypothesis_testing': True,
            'best_k_selected_from_internal_unsupervised_criteria': True,
        },
    }

view_summaries = {
    'disease_only': evaluate_view('disease_only'),
    'all_subjects': evaluate_view('all_subjects'),
    'binary_pathological_vs_control':
↪evaluate_view('binary_pathological_vs_control'),
    'control_a_vs_control_b': evaluate_view('control_a_vs_control_b'),
}

X_stride = stride_df[FEATURE_COLS].to_numpy(dtype=float)
conditions = stride_df['condition'].to_numpy()
labels = stride_df['label'].to_numpy()
cond_label = np.array([f"{c}_{'disease' if y == 1 else 'control'}" for c, y in
↪zip(conditions, labels)])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_stride)
pca = PCA(n_components=len(FEATURE_COLS), random_state=RANDOM_SEEDS['pca'])
X_pca = pca.fit_transform(X_scaled)
explained_var = pca.explained_variance_ratio_
cumulative_var = np.cumsum(explained_var)

fig, ax1 = plt.subplots(figsize=(IEEE_ONE_COL * 1.7, 2.9),
↪constrained_layout=True)
components = np.arange(1, len(explained_var) + 1)
ax1.bar(components, explained_var, color='#4c72b0', alpha=0.75)
ax1.set_xlabel('Principal component')
ax1.set_ylabel('Explained variance ratio')

```

```

ax2 = ax1.twinx()
ax2.plot(components, cumulative_var, color='#dd8452', marker='o', linewidth=1.2)
ax2.axhline(0.90, color='gray', linestyle='--', linewidth=1)
ax2.axhline(0.95, color='black', linestyle=':', linewidth=1)
ax2.set_ylabel('Cumulative explained variance')
ax2.set_ylim(0.0, 1.02)
ax1.set_title('PCA explained variance (stride-level v4 matrix)')
savefig_stem(fig, 'pca_explained_variance')
plt.show()

pca_plot_df = pd.DataFrame({
    'PC1': X_pca[:, 0],
    'PC2': X_pca[:, 1],
    'condition': conditions,
    'label': labels,
})

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.85, 3.5),
    constrained_layout=True)
for cond in COND_ORDER:
    subset = pca_plot_df[pca_plot_df['condition'] == cond]
    if subset.empty:
        continue
    dis = subset[subset['label'] == 1]
    ctl = subset[subset['label'] == 0]
    if len(dis):
        ax.scatter(dis['PC1'], dis['PC2'], s=14, alpha=0.35,
            color=COND_COLOR[cond], marker='o', label=f"{COND_LABEL[cond]} disease")
    if len(ctl):
        ax.scatter(ctl['PC1'], ctl['PC2'], s=22, alpha=0.65, facecolors='none',
            edgecolors=COND_COLOR[cond], linewidths=0.8, marker='o',
            label=f"{COND_LABEL[cond]} control")
ax.set_title('PCA Projection: Condition Separability (PC1 vs PC2)')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.grid(alpha=0.2)
ax.legend(loc='upper right', fontsize=7, ncol=2, frameon=True)
savefig_stem(fig, 'pca_scatter_by_condition')
plt.show()

loadings = pd.DataFrame(pca.components_[1:2].T, index=FEATURE_COLS,
    columns=['PC1', 'PC2'])
save_table_csv(loadings, 'step6_pca_feature_loadings.csv', index=True)

k_values = list(range(2, 11))
inertias = []
sil_scores = []

```

```

clusters_by_k = {}
for k in k_values:
    km = KMeans(n_clusters=k, random_state=RANDOM_SEEDS['pca'], n_init=20)
    preds = km.fit_predict(X_scaled)
    clusters_by_k[k] = preds
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, preds))

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.45, 2.7),
    constrained_layout=True)
ax.plot(k_values, inertias, marker='o', color='#4c72b0')
ax.set_title('K-means elbow curve (stride-level v4 matrix)')
ax.set_xlabel('Number of clusters (K)')
ax.set_ylabel('Inertia')
ax.grid(alpha=0.2)
savefig_stem(fig, 'kmeans_elbow')
plt.show()

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.45, 2.7),
    constrained_layout=True)
ax.plot(k_values, sil_scores, marker='o', color='#dd8452')
ax.set_title('K-means silhouette scores (stride-level v4 matrix)')
ax.set_xlabel('Number of clusters (K)')
ax.set_ylabel('Silhouette score')
ax.grid(alpha=0.2)
savefig_stem(fig, 'kmeans_silhouette')
plt.show()

clusters3 = clusters_by_k[3]
cluster_plot_df = pca_plot_df.copy()
cluster_plot_df['cluster_k3'] = clusters3.astype(int)
fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.85, 3.5),
    constrained_layout=True)
scatter = ax.scatter(cluster_plot_df['PC1'], cluster_plot_df['PC2'],
    c=cluster_plot_df['cluster_k3'], cmap='tab10', s=16, alpha=0.5)
ax.set_title('K-Mean (K=3) Overlay in PCA Space')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.grid(alpha=0.2)
legend = ax.legend(*scatter.legend_elements(), title='Cluster', loc='upper_
    right', frameon=True)
ax.add_artist(legend)
savefig_stem(fig, 'kmeans_scatter_k3')
plt.show()

summary_payload = {
    'protocol_manifest_sha256': PREPROC['protocol_manifest_sha256'],

```

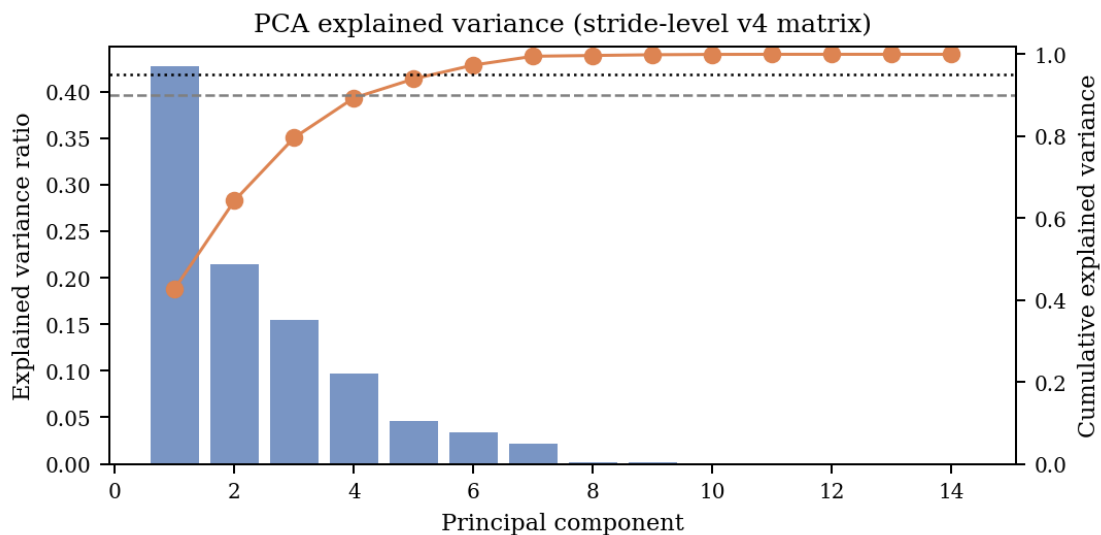
```

'preprocessing_manifest_sha256': sha256_file(PREPROC_PATH),
'downstream_execution_manifest_sha256': sha256_file(DOWNSTREAM_PATH),
'subject_level_matrix_sha256': PREPROC['subject_level_matrix_sha256'],
'notebook_source_sha256': sha256_file(NOTEBOOK_PATH),
'package_versions': collect_package_versions(),
'random_seeds': RANDOM_SEEDS,
'views': view_summaries,
}
save_json_payload(summary_payload, SUMMARY_JSON)

display(pd.DataFrame([
    {'view': key, 'subject_count': value['subject_count'], 'silhouette_score':
    ↪value['silhouette_score'], 'ari_against_relevant_labels':
    ↪value['ari_against_relevant_labels']}
    for key, value in view_summaries.items()
]).sort_values('silhouette_score', ascending=False))

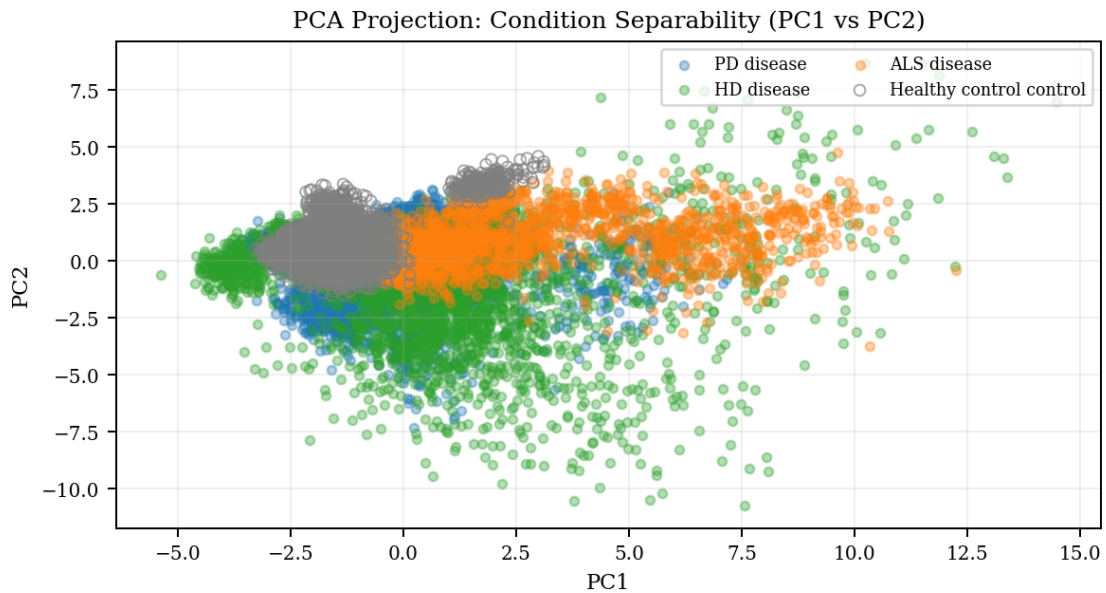
```

Saved table: report/tables/v4/step6_disease_only_pca_coordinates.csv
 Saved table: report/tables/v4/step6_disease_only_kmeans_metrics.csv
 Saved table: report/tables/v4/step6_all_subjects_pca_coordinates.csv
 Saved table: report/tables/v4/step6_all_subjects_kmeans_metrics.csv
 Saved table:
 report/tables/v4/step6_binary_pathological_vs_control_pca_coordinates.csv
 Saved table:
 report/tables/v4/step6_binary_pathological_vs_control_kmeans_metrics.csv
 Saved table: report/tables/v4/step6_control_a_vs_control_b_pca_coordinates.csv
 Saved table: report/tables/v4/step6_control_a_vs_control_b_kmeans_metrics.csv
 Saved figure: report/figures/v4/pdf/step6/pca_explained_variance.pdf
 Saved figure: report/figures/v4/png/step6/pca_explained_variance.png



Saved figure: report/figures/v4/pdf/step6/pca_scatter_by_condition.pdf

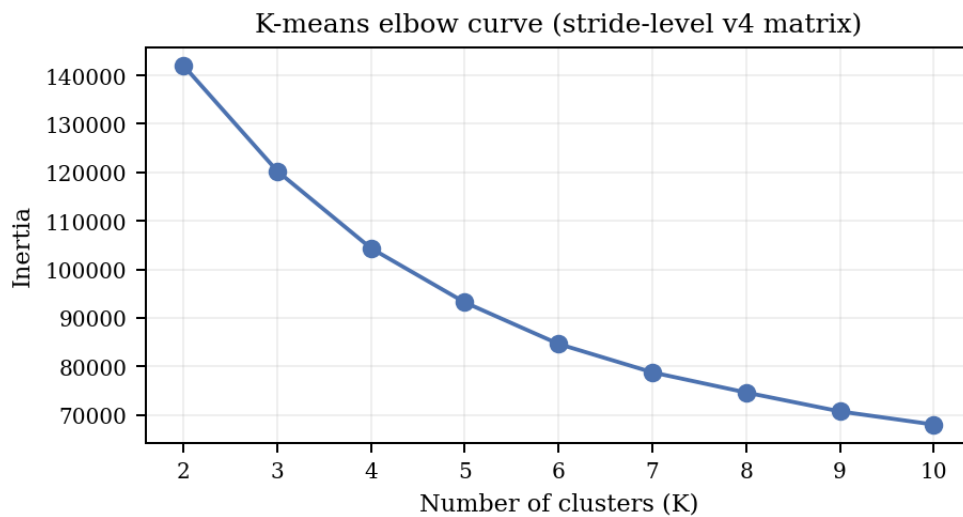
Saved figure: report/figures/v4/png/step6/pca_scatter_by_condition.png



Saved table: report/tables/v4/step6_pca_feature_loadings.csv

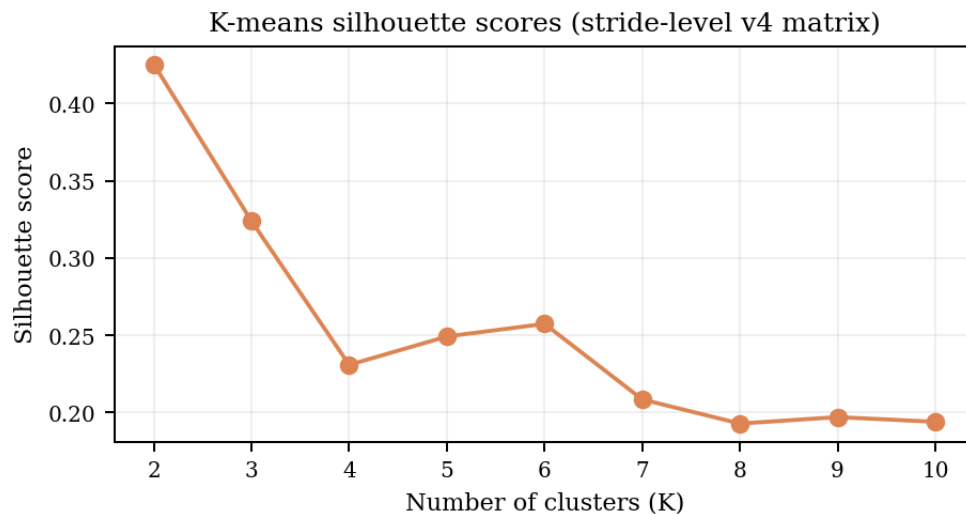
Saved figure: report/figures/v4/pdf/step6/kmeans_elbow.pdf

Saved figure: report/figures/v4/png/step6/kmeans_elbow.png



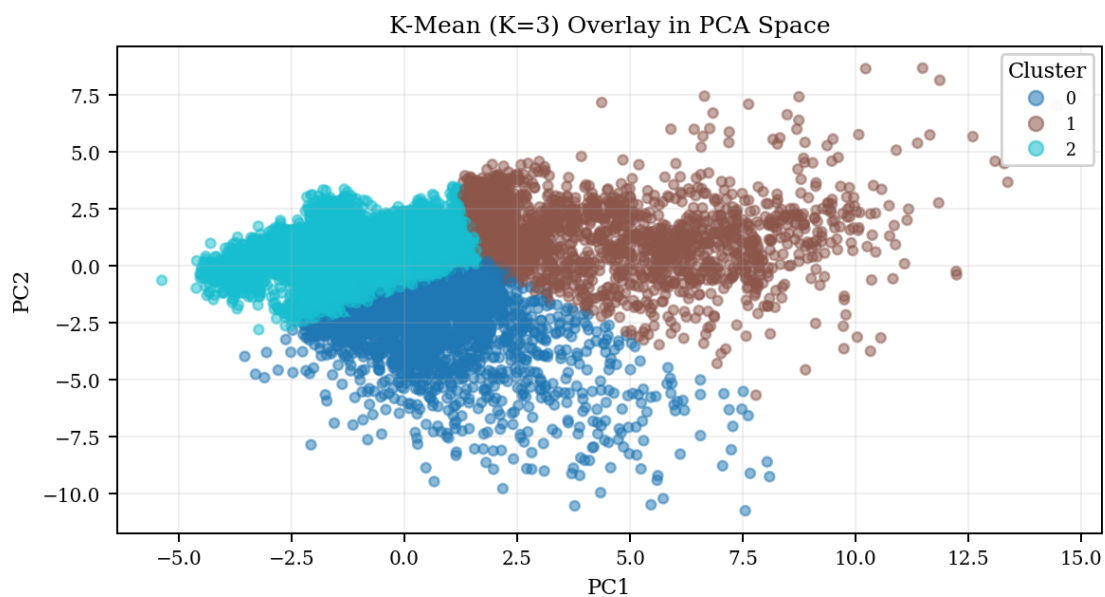
Saved figure: report/figures/v4/pdf/step6/kmeans_silhouette.pdf

Saved figure: report/figures/v4/png/step6/kmeans_silhouette.png



Saved figure: report/figures/v4/pdf/step6/kmeans_scatter_k3.pdf

Saved figure: report/figures/v4/png/step6/kmeans_scatter_k3.png



Wrote summary: experiments/results/v4/step6_subject_views_summary.json

	view	subject_count	silhouette_score	\
1	all_subjects	63	0.490944	
2	binary_pathological_vs_control	63	0.490944	
3	control_a_vs_control_b	16	0.487732	
0	disease_only	47	0.462493	

	ari_against_relevant_labels
1	0.025964
2	-0.099877
3	0.000000
0	0.013473

1.2 Interpretation notes

The Step 6 figures are useful because they provide qualitative context for why some supervised transfer directions may be easier or harder, but they do **not** prove unsupervised disease separability. The saved summary also preserves seed-stability and subject-bootstrap stability metrics, which matter because a visually appealing cluster plot is not enough to justify a stronger claim. The safest reading is that the subject-level feature space contains visible structure, while the label alignment of that structure remains too weak for confirmatory interpretation.